⚡ DATABRICKS SPARK SQL CHEAT SHEET

Production Reference: DDL, DML, Liquid Clustering, Delta Lake ACID & Medallion Patterns

SPARK SQL | DBR 15.4+ LTS | PHOTON ENGINE | UNITY CATALOG

🏛️ Unity Catalog 3-Level Namespace & DDL

Governance

```sql
-- 3-Level Hierarchy: catalog.schema.table
CREATE CATALOG IF NOT EXISTS prod_lakehouse;
CREATE SCHEMA IF NOT EXISTS prod_lakehouse.silver_finance
  MANAGED LOCATION 'abfss://silver@storage.dfs.core.windows.net/';
USE CATALOG prod_lakehouse; USE SCHEMA silver_finance;

-- Liquid Clustering (Modern replacement for PARTITION BY)
CREATE TABLE transactions (
  txn_id BIGINT GENERATED ALWAYS AS IDENTITY,
  customer_id STRING,
  amount DECIMAL(18,2),
  txn_date DATE
) USING DELTA CLUSTER BY (txn_date, customer_id);

-- Zero-Copy Shallow Clone (instant pointer for dev/staging)
CREATE TABLE dev_txns SHALLOW CLONE transactions;
-- Deep Clone (full independent storage copy for DR/backup)
CREATE TABLE dr_backup DEEP CLONE transactions;
```

💡 **Liquid Clustering:** Avoids small-file skew, supports mutable keys, and re-clusters incrementally without full partition rewrites.

🔄 Delta Lake ACID Upsert (MERGE INTO)

DML / Medallion

```sql
MERGE INTO silver.customers AS target
USING bronze.stg_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.is_deleted = true THEN
  DELETE
WHEN MATCHED AND target.hash_diff ≠ source.hash_diff THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, status, created_at, updated_at)
  VALUES (source.customer_id, source.email, source.status,
          current_timestamp(), current_timestamp())
WHEN NOT MATCHED BY SOURCE AND target.status = 'ACTIVE' THEN
  UPDATE SET target.status = 'INACTIVE';
```

⚡ **Performance:** Always provide deterministic partition or cluster keys in the `ON` clause to enable Delta Dynamic File Pruning (DFP).

🛠️ Compaction, Z-Order & Vacuum

FinOps / Performance

```sql
-- Compact small files into ~1GB Parquet target files
OPTIMIZE silver.transactions;

-- Multidimensional Space-Filling Z-Order (Legacy/Static tables)
OPTIMIZE silver.transactions ZORDER BY (customer_id, txn_date);

-- Purge unreferenced historical parquet files older than 7 days
VACUUM silver.transactions RETAIN 168 HOURS;

-- Collect CBO statistics for adaptive join planning
ANALYZE TABLE silver.transactions COMPUTE STATISTICS FOR ALL COLUMNS;
```

⚠️ **Vacuum Caution:** Never set retention < 168 hours in production. Vacuuming 0 hours breaks concurrent active readers and invalidates time travel.

⏳ Delta Time Travel, Audit & Instant Rollback

Resilience

```sql
-- Query historical snapshot by version integer
SELECT * FROM silver.transactions VERSION AS OF 14;

-- Query snapshot by UTC timestamp string
SELECT * FROM silver.transactions
TIMESTAMP AS OF '2026-10-15 08:30:00';

-- Instant point-in-time recovery (Disaster Rollback)
RESTORE TABLE silver.transactions TO VERSION AS OF 13;

-- Inspect full commit history & user audit trail
DESCRIBE HISTORY silver.transactions LIMIT 10;
```

- `DESCRIBE HISTORY` records exact user, notebook ID, cluster ID, and metrics.
- `RESTORE TABLE` takes milliseconds — updates Delta transaction log pointer.

🔗 Chained CTEs, Window Slicing & Set Operations

Core SQL

```sql
WITH raw_txns AS (
  SELECT customer_id, region, amount,
         DENSE_RANK() OVER(PARTITION BY region ORDER BY amount DESC) AS rk
  FROM silver.transactions WHERE txn_date ≥ current_date() - 30
),
top_spenders AS (
  SELECT customer_id, region, amount FROM raw_txns WHERE rk ≤ 10
)
SELECT * FROM top_spenders
UNION ALL
SELECT customer_id, region, amount FROM silver.manual_overrides
EXCEPT DISTINCT
SELECT customer_id, region, amount FROM silver.fraud_blacklist;
```

💡 **Recursive CTE:** Use `WITH RECURSIVE` to traverse bill-of-materials or organizational hierarchies in a single SQL pass.

🥇 Medallion SQL & QUALIFY Deduplication

Architecture

```sql
-- Silver Cleanse: QUALIFY filter eliminates nested subqueries!
CREATE OR REPLACE TABLE silver.orders_cleansed AS
SELECT
  order_id, customer_id, amount,
  CAST(order_date AS DATE) AS order_date,
  _metadata.file_modification_time AS ingested_at
FROM bronze.orders_raw
WHERE amount > 0 AND order_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER(
  PARTITION BY order_id ORDER BY ingested_at DESC
) = 1;

-- Gold Aggregation: Pre-computed dimension metrics
CREATE OR REPLACE TABLE gold.customer_metrics AS
SELECT customer_id, COUNT(*) AS orders, SUM(amount) AS ltv
FROM silver.orders_cleansed GROUP BY customer_id;
```

⚡ **QUALIFY Clause:** Replaces expensive `SELECT * FROM (SELECT ..., ROW_NUMBER() ...) WHERE rn=1` subqueries cleanly!

⚡ DATABRICKS SPARK SQL CHEAT SHEET

Analytical SQL, Complex Types, Higher-Order Functions, Pivot & Engine Tuning

ADVANCED SQL | ANALYTICAL FUNCTIONS | ROW FILTERS / MASKS

📊 Window Functions & Analytical Framing

Analytics

```sql
SELECT customer_id, txn_date, amount,
  -- Ranking: Row_Number (unique), Dense_Rank (no gaps)
  ROW_NUMBER() OVER(w_acct ORDER BY txn_date DESC) AS rn,
  DENSE_RANK() OVER(w_acct ORDER BY amount DESC) AS rk,
  -- Offsets: Prior & Next transaction amount
  LAG(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS prev_amt,
  LEAD(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS next_amt,
  -- Moving 7-Day Rolling Average (Physical rows)
  AVG(amount) OVER(w_acct ORDER BY txn_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS roll_7d_avg,
  -- Cumulative Year-to-Date Sum (Range)
  SUM(amount) OVER(w_acct ORDER BY txn_date
    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ytd_sum
FROM silver.transactions
WINDOW w_acct AS (PARTITION BY customer_id);
```

Pro Tip: If source data is clustered/sorted by the window partition key, Catalyst executes windowing without any network shuffle!

🧩 Complex Types: Structs, Arrays & JSON

Semi-Structured

```sql
-- Struct navigation, Array slicing, Map lookup
SELECT
  payload.user.id AS user_id,              -- struct field
  payload.tags[0] AS primary_tag,          -- array 0-index
  payload.attributes['tier'] AS tier,      -- map lookup
  -- Parse raw JSON string into typed Struct
  from_json(raw_payload, 'id BIGINT, name STRING, items ARRAY<STRING>') AS parsed,
  -- Convert Struct into JSON string
  to_json(named_struct('ts', current_timestamp(), 'code', 200)) AS json_str
FROM bronze.events;

-- Explode Array into relational rows
SELECT order_id, item.sku, item.qty, item.unit_price
FROM silver.orders
LATERAL VIEW EXPLODE(line_items) lv AS item;
```

- `LATERAL VIEW OUTER EXPLODE` preserves rows with empty/null arrays.
- `schema_of_json()` dynamically extracts schema from a sample JSON string.

⚡ Higher-Order Functions (In-Engine Array Ops)

Photon Vectorized

```sql
SELECT order_id,
  -- TRANSFORM: Map elements without exploding!
  TRANSFORM(line_items, x ->
    named_struct('sku', x.sku, 'gross', ROUND(x.price * 1.20, 2))
  ) AS taxed_items,

  -- FILTER: Retain only elements matching condition
  FILTER(line_items, x -> x.price > 100.0) AS luxury_items,

  -- EXISTS: Returns true if any element satisfies predicate
  EXISTS(line_items, x -> x.sku LIKE 'PROMO-%') AS has_promo,

  -- AGGREGATE: Reduce array to scalar sum
  AGGREGATE(line_items, 0.0, (acc, x) -> acc + x.price,
            acc -> ROUND(acc, 2)) AS total_order_sum
FROM silver.orders;
```

Architect Rule: Higher-Order functions run entirely in memory inside Tungsten/Photon C++ without row explosion or shuffle overhead.

🔒 Unity Catalog Row Filters & Column Masks

Security

```sql
-- 1. Column Masking (Mask card PAN unless in finance)
CREATE OR REPLACE FUNCTION mask_card_pan(pan STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('pci_auditors'), pan,
          CONCAT('****-****-****-', RIGHT(pan, 4)));

ALTER TABLE silver.payments
ALTER COLUMN card_pan SET MASK mask_card_pan;

-- 2. Row Filtering (Regional multi-tenant isolation)
CREATE OR REPLACE FUNCTION filter_by_region(region STRING)
RETURN IS_ACCOUNT_GROUP_MEMBER('global_admin') OR region = current_user();

ALTER TABLE silver.sales
SET ROW FILTER filter_by_region ON (region);
```

Fine-Grained Security: Applied transparently at storage read-time for BI, SQL, Python, and R without maintaining separate secure views.

🔄 PIVOT, UNPIVOT & Grouping Sets

Transformations

```sql
-- PIVOT: Rotate row values into columnar KPIs
SELECT * FROM (
  SELECT region, quarter, revenue FROM silver.sales
) PIVOT (
  SUM(revenue) FOR quarter IN ('Q1', 'Q2', 'Q3', 'Q4')
);

-- UNPIVOT: Rotate columnar metrics into tidy rows
SELECT * FROM silver.quarterly_kpis
UNPIVOT (revenue FOR qtr IN (q1, q2, q3, q4));

-- CUBE & ROLLUP: Multi-level executive aggregation
SELECT region, product, SUM(revenue) AS total
FROM silver.sales GROUP BY ROLLUP (region, product);
```

ROLLUP vs CUBE: `ROLLUP` produces hierarchical subtotals (Country > State > City); `CUBE` computes all 2^N combinations.

🚀 Join Strategy Hints & Session Performance Configuration

Engine Tuning

```sql
-- Optimizer Join Hints
SELECT /*+ BROADCAST(dim_cat) */
  f.txn_id, f.amount, dim_cat.category_name
FROM silver.fact_sales f
JOIN silver.dim_category dim_cat ON f.category_id = dim_cat.id;

-- Force Shuffle Hash Join
SELECT /*+ SHUFFLE_HASH(orders) */ *
FROM orders JOIN returns ON orders.id = returns.id;
```

Knob	Setting	Impact
`spark.sql.shuffle.partitions`	`auto`	Enables AQE dynamic partition sizing
`spark.sql.adaptive.enabled`	`true`	Runtime join re-planning & skew split
`spark.databricks.delta.optimizeWrite.enabled`	`true`	Pre-shuffle writes to prevent small files